

Computer Architecture

CE/CS 321/330

L2

Course Teacher: Dr.-Ing. Shehzad Hasan

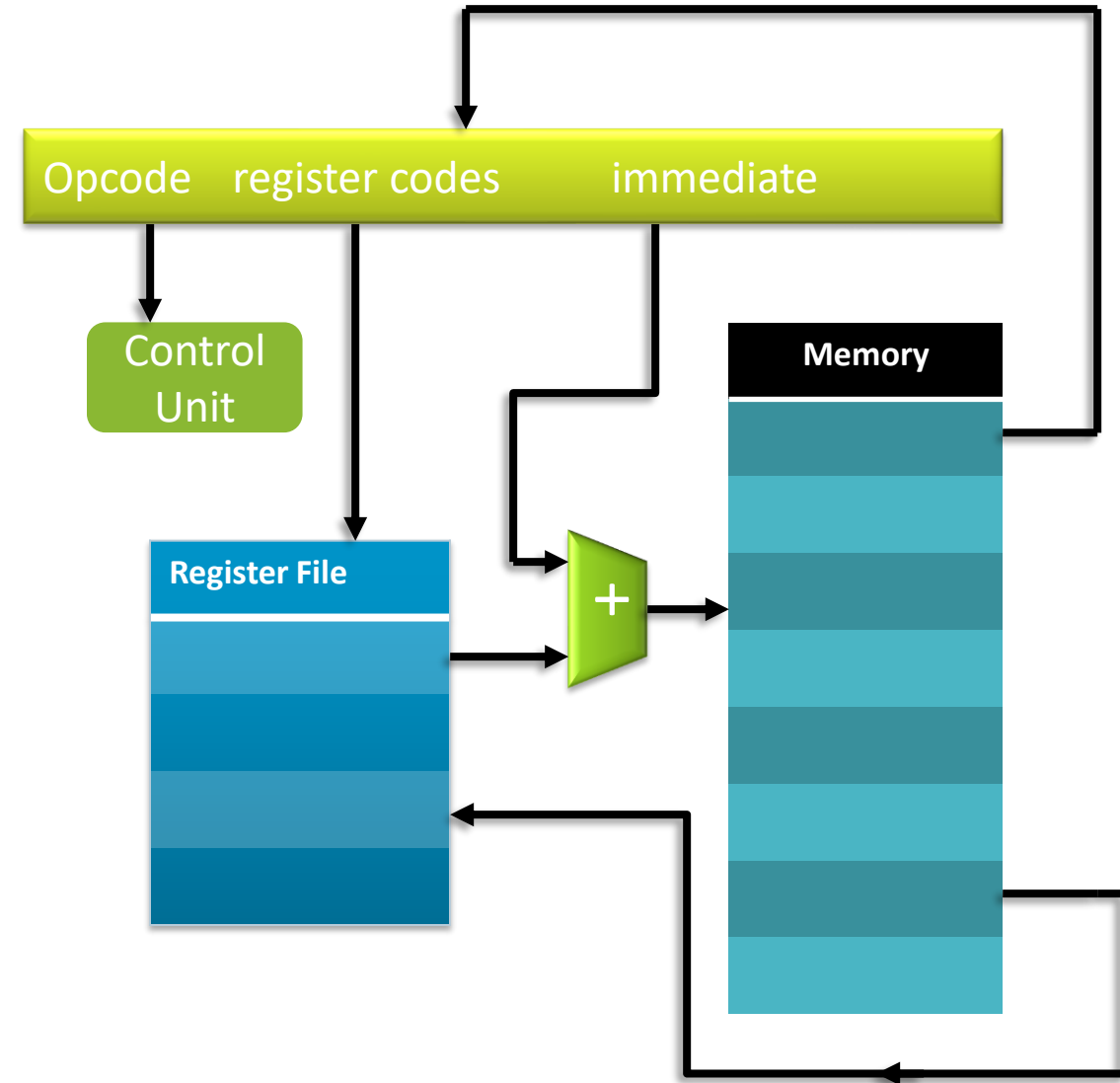
Lecture: 18

Instruction Pipelining

- Instruction cycle for a Load Instruction

ld x10, 100 (x12)

- 1) Instruction is fetched
- 2) Instruction is decoded +
Source register contents are fetched
- 3) Address is calculated by adding the immediate address field with the source register contents
- 4) Data is fetched from memory from that address
- 5) The fetched data is stored in the destination register



Instruction Pipelining

Load Instruction Steps

1) Instruction is fetched

2) a) Instruction is decoded
b) Source register contents are fetched

3) Address is calculated by adding the immediate address field with the source register contents

4) Data is fetched from memory from that address

5) The fetched data is stored in the destination register

1) IF – Instruction Fetch

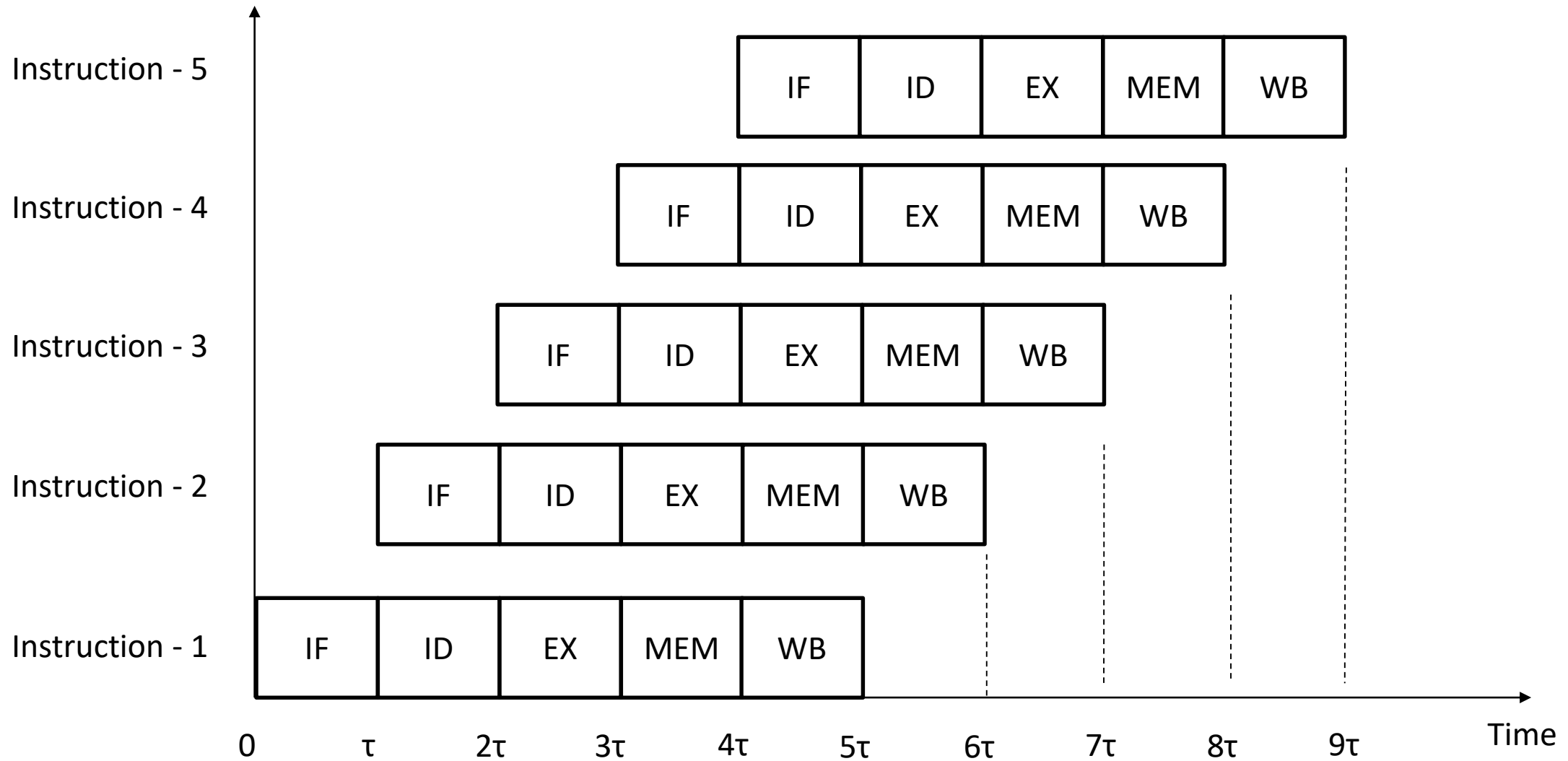
2) ID – Instruction Decode

3) EX – Execute

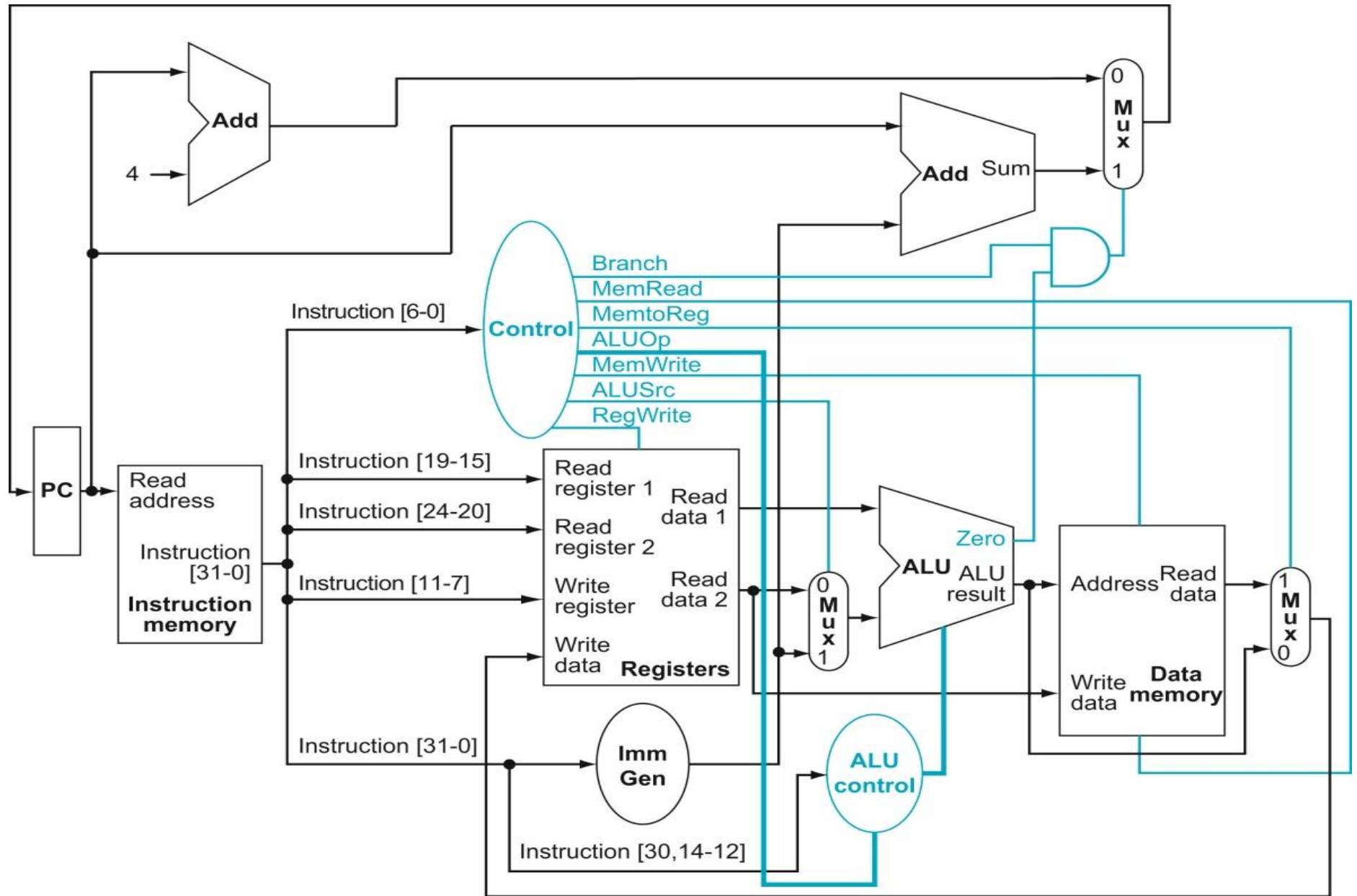
4) MEM – Memory

5) WB – Write Back

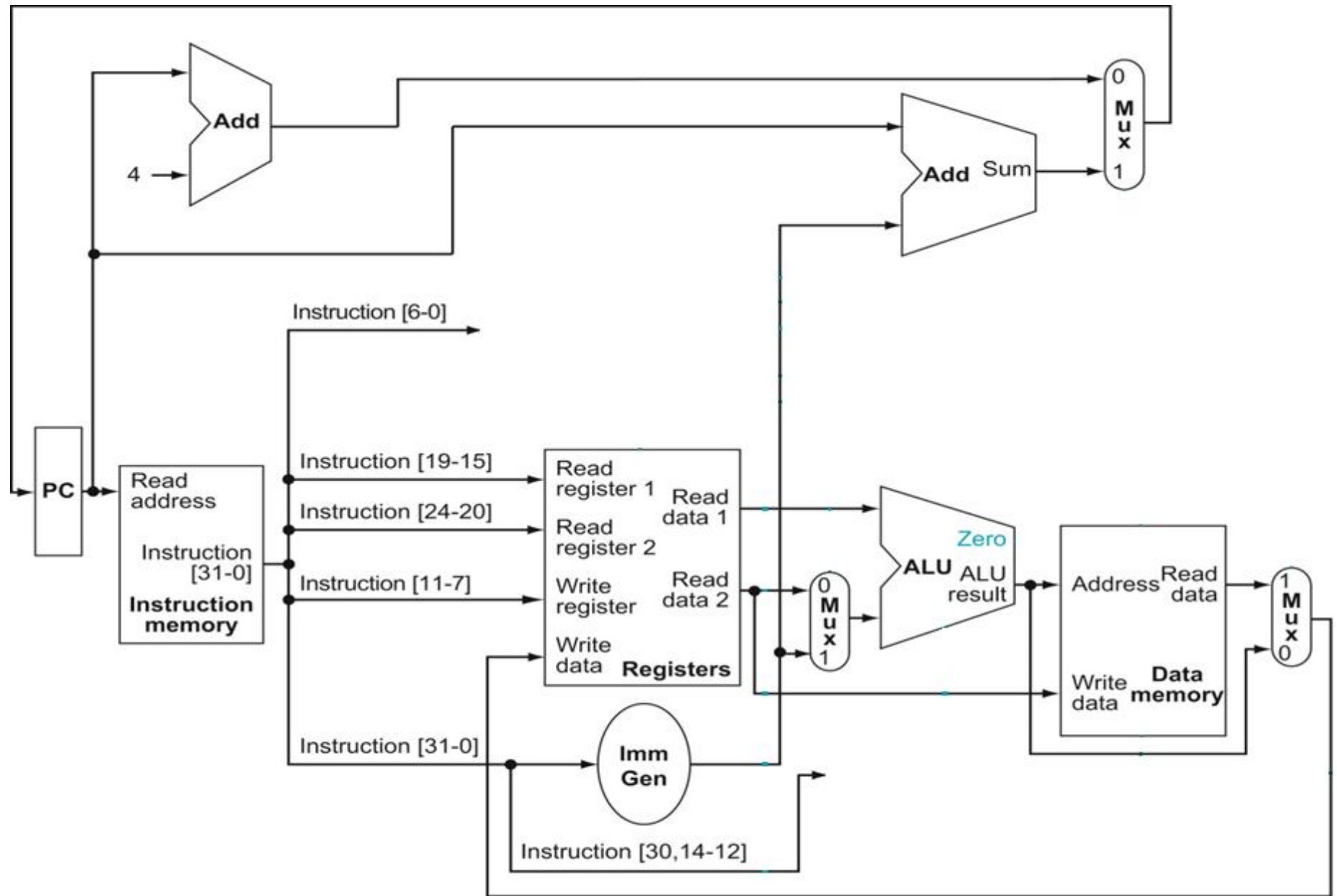
Instruction Pipelining



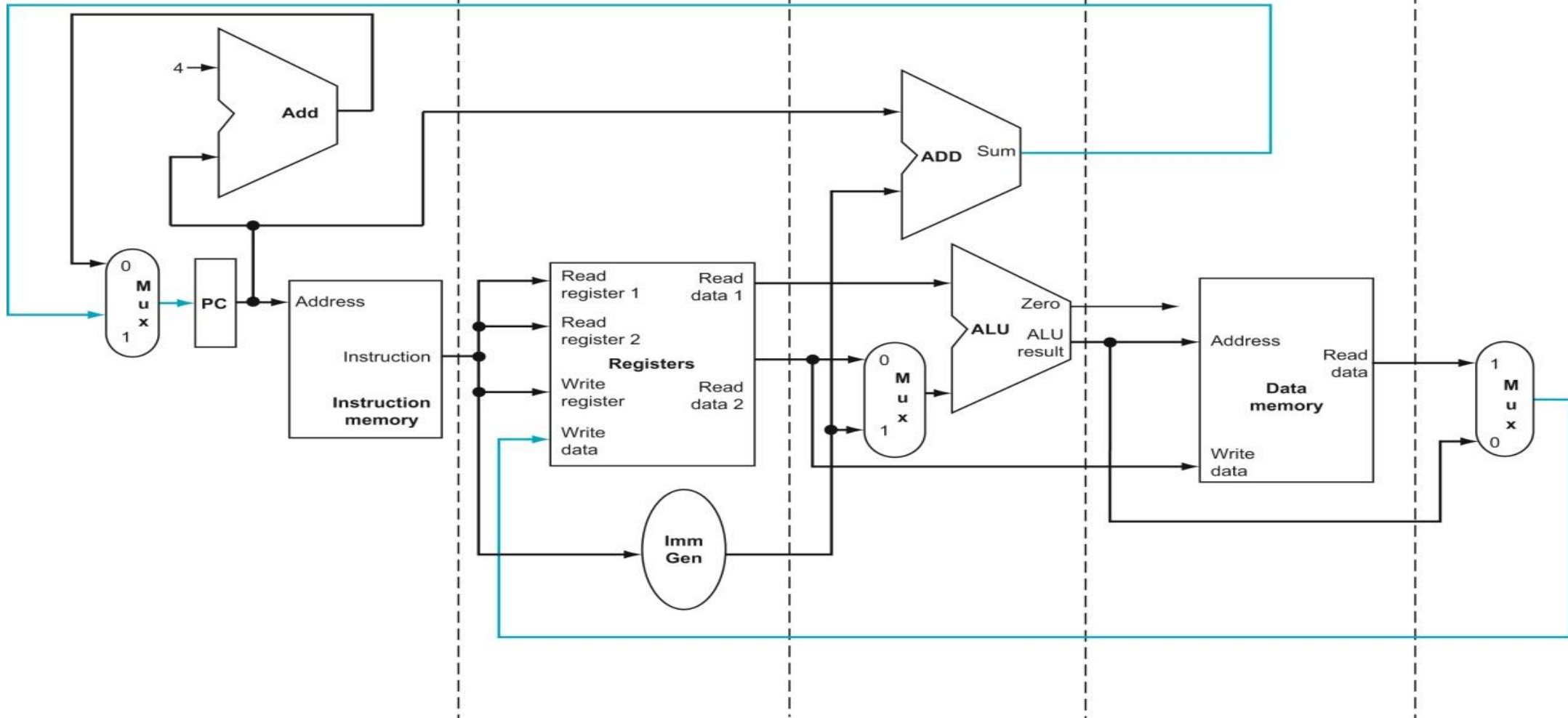
Single Cycle RISC-V Datapath + Control



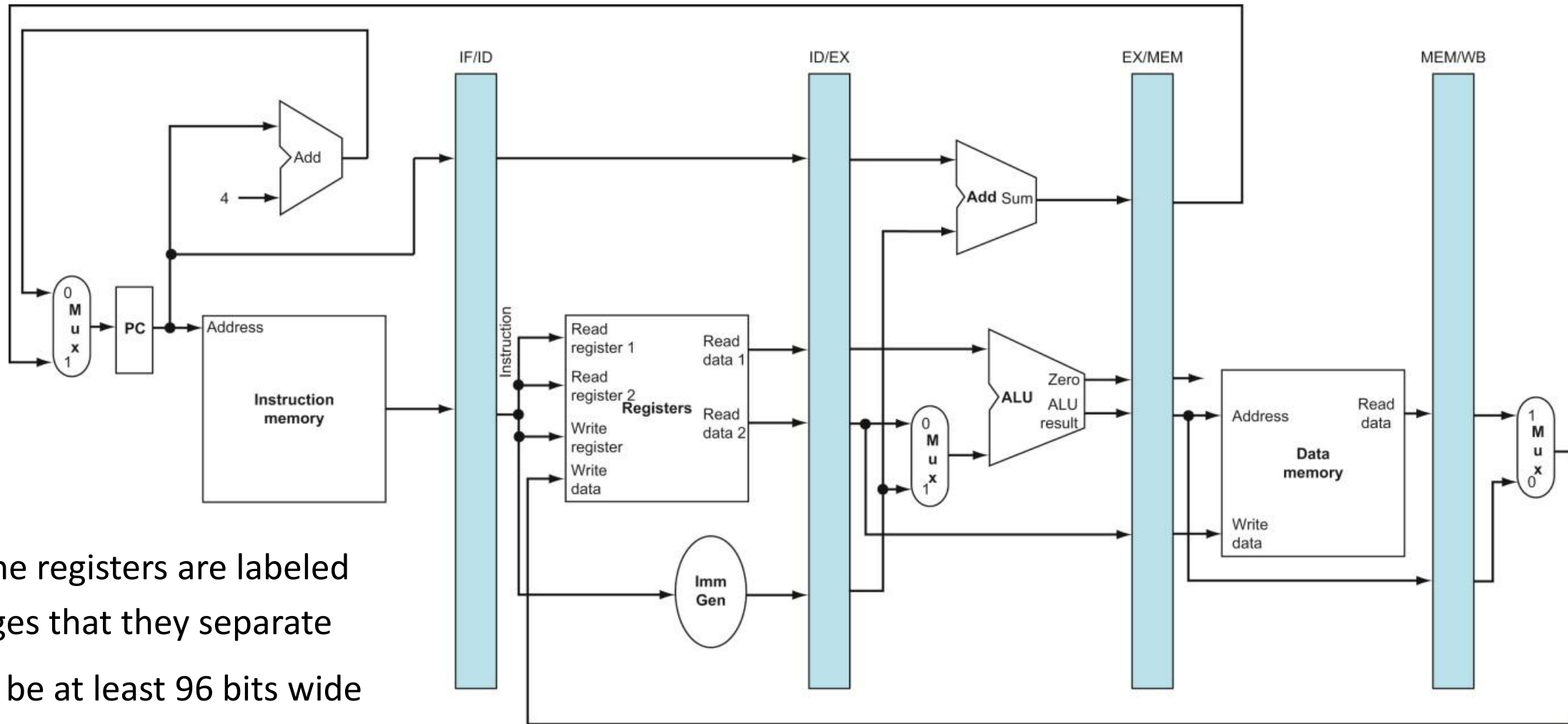
Single Cycle RISC-V Datapath



IF – Instruction Fetch ID – Instruction Decode
 EX – Execute MEM – Memory WB – Write Back



Pipelined Datapath

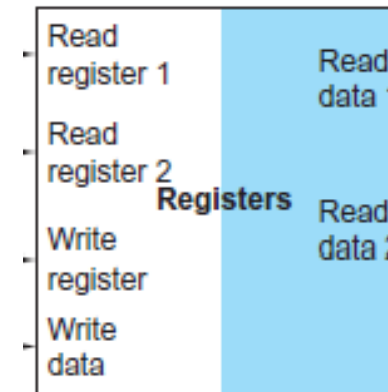


- The pipeline registers are labeled by the stages that they separate
- IF/ID must be at least 96 bits wide to hold recently fetched instruction and PC values

Load Instruction

Let's see how the load instruction goes through the pipelined datapath and see if everything works fine

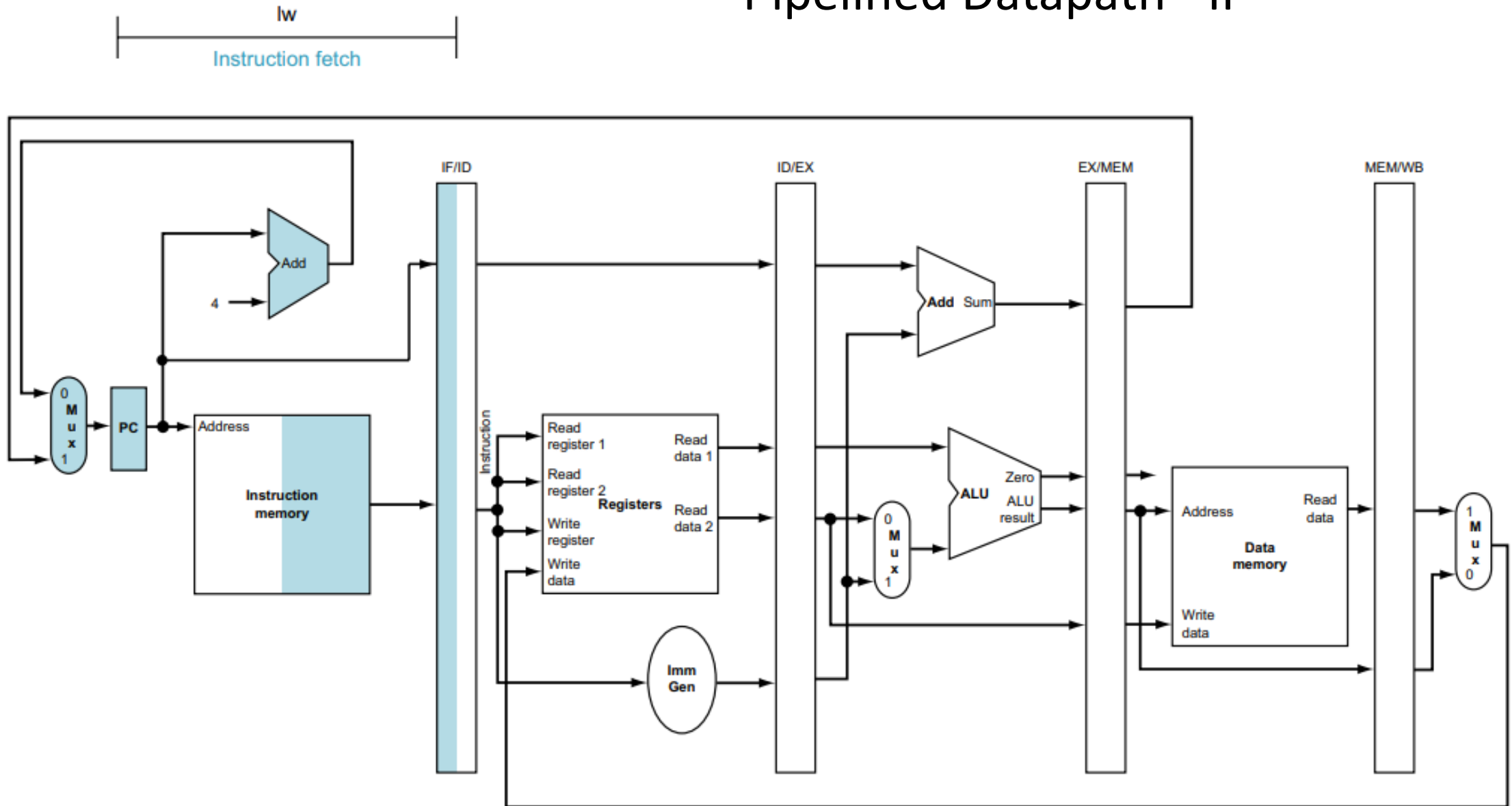
- The *right half* of registers or memory is highlighted when they are being *read*
- The *left half* of registers or memory is highlighted when they are being *written*



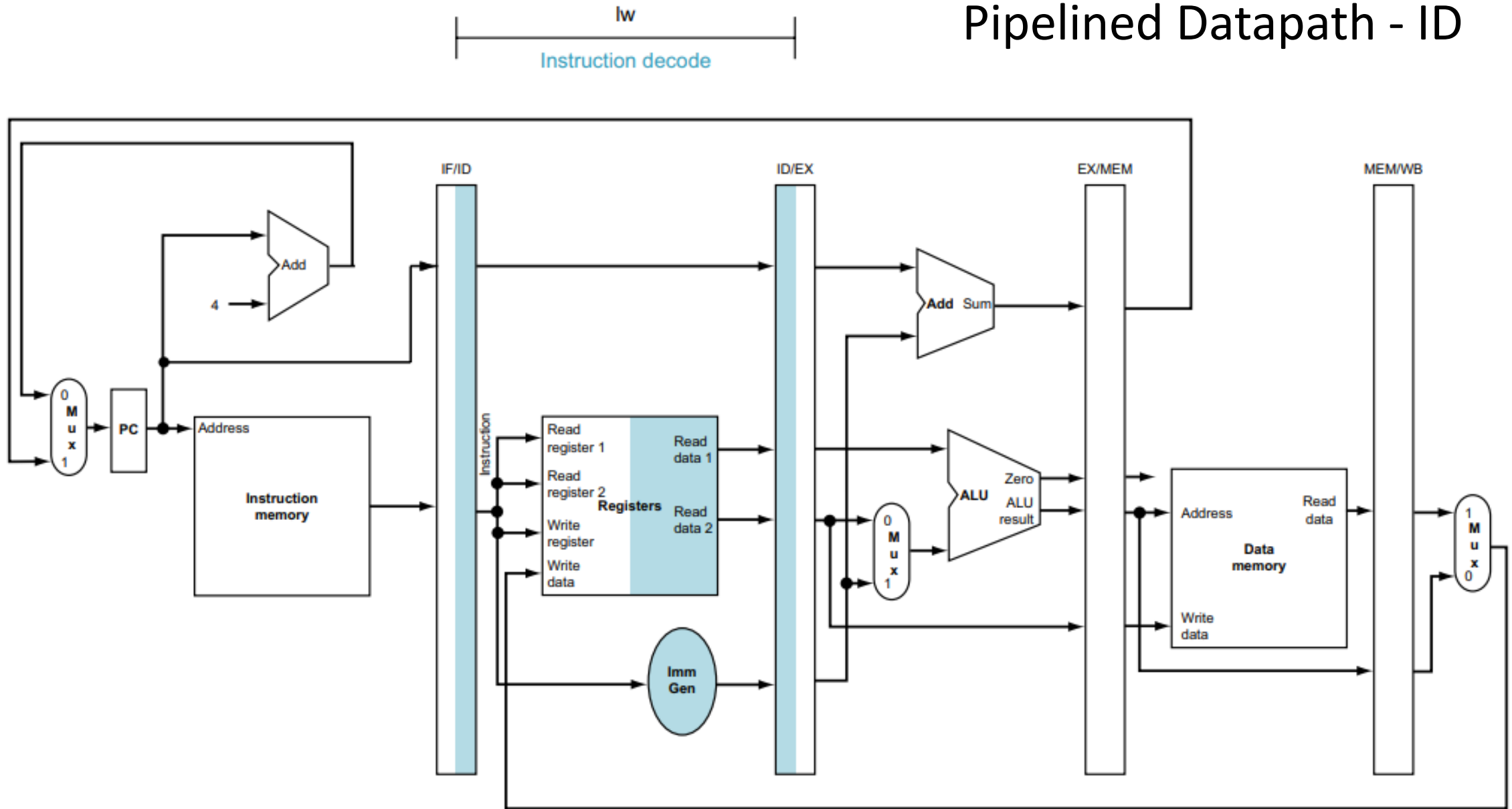
IF/ID



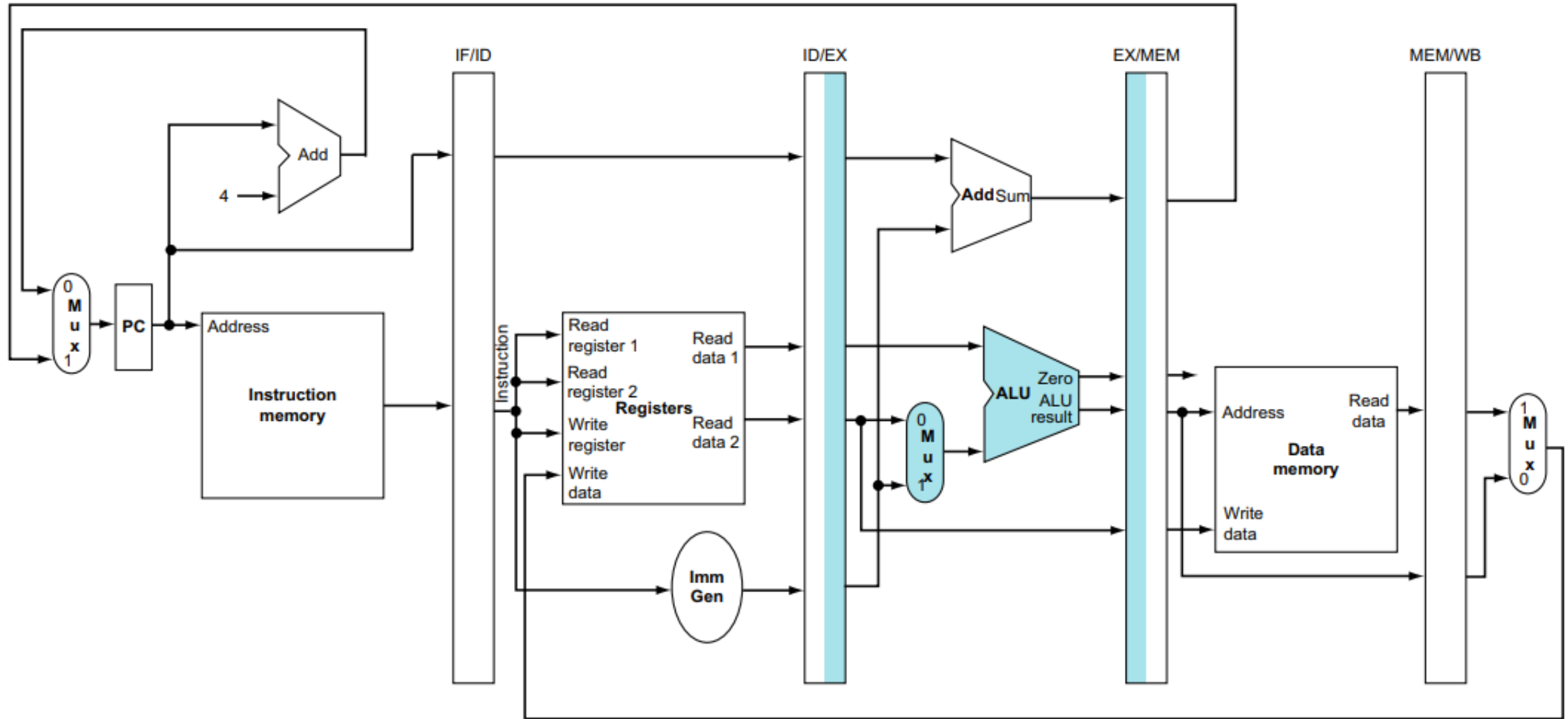
Pipelined Datapath - IF



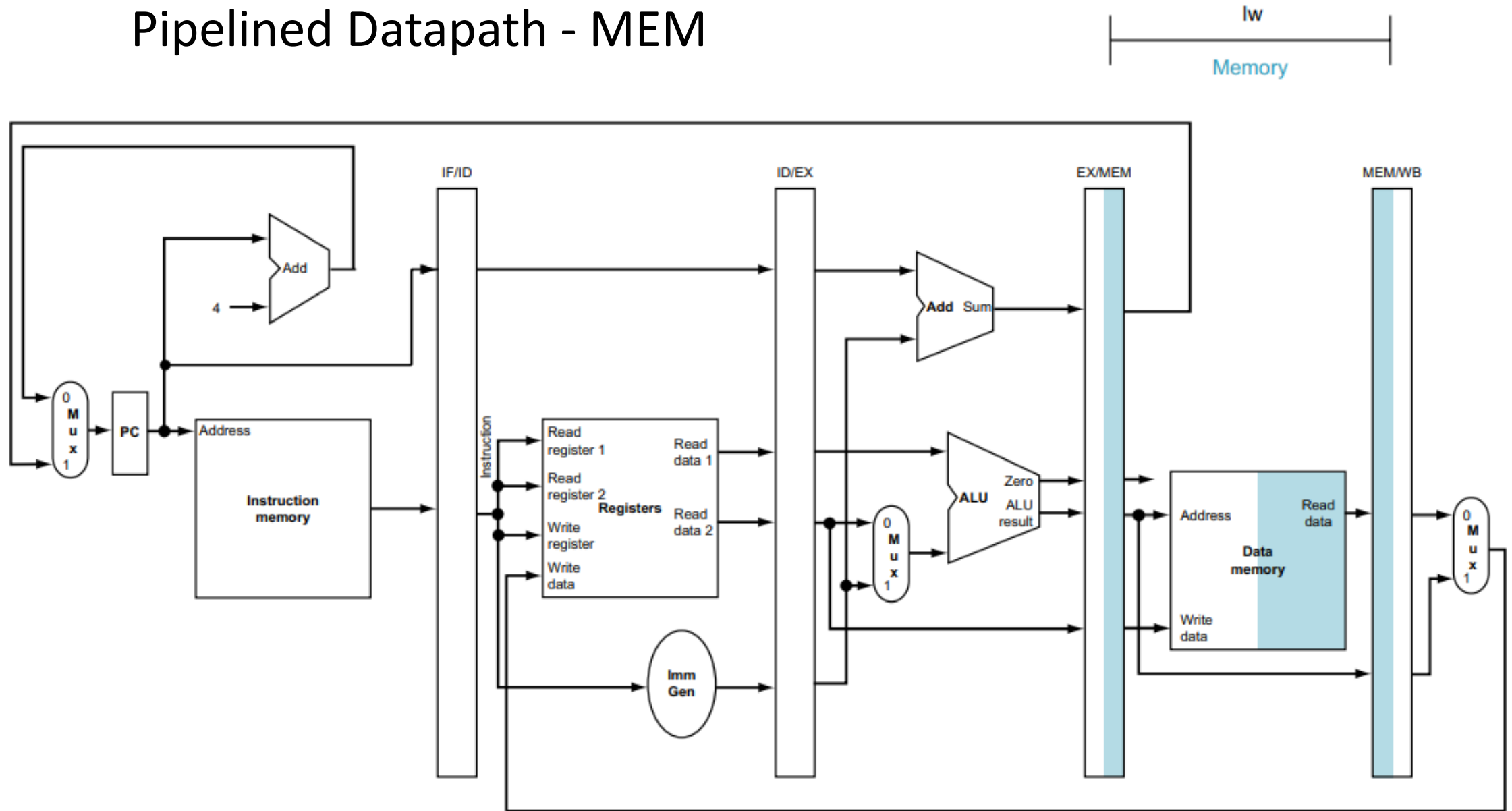
Pipelined Datapath - ID



Pipelined Datapath - EX

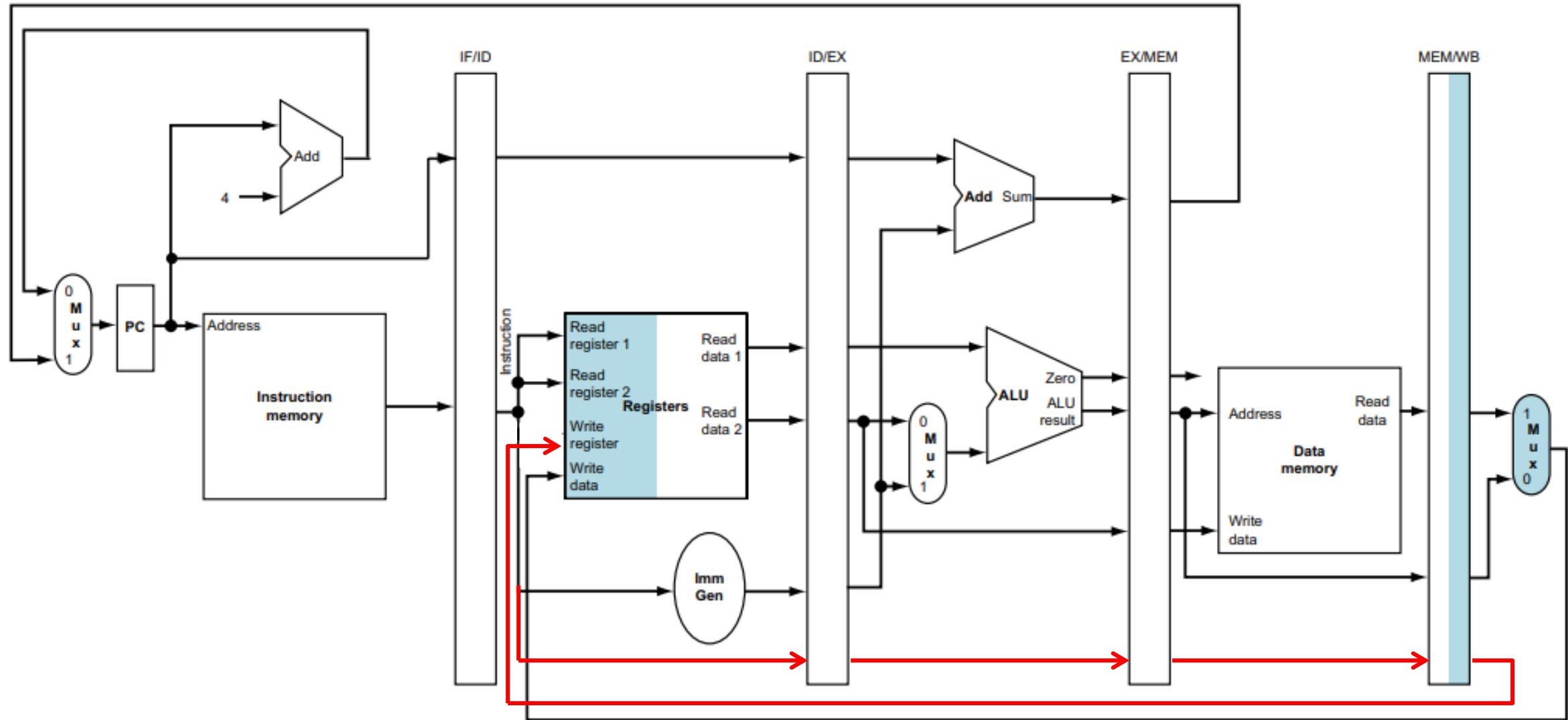


Pipelined Datapath - MEM

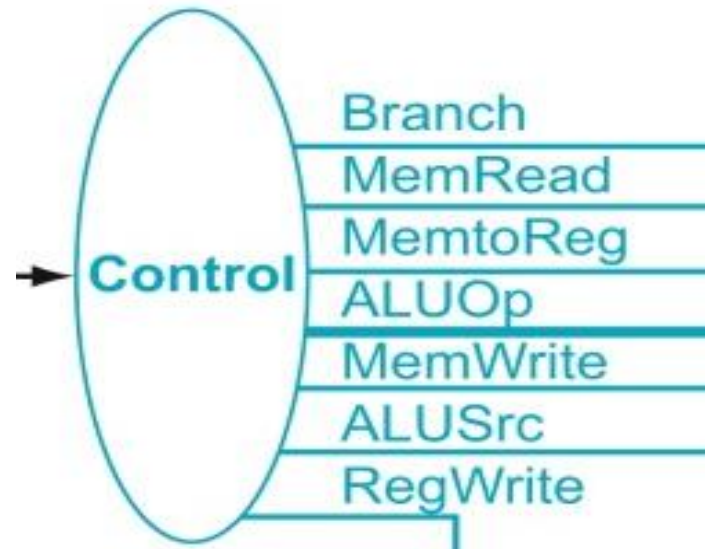


Pipelined Datapath - WB

lw
Write-back

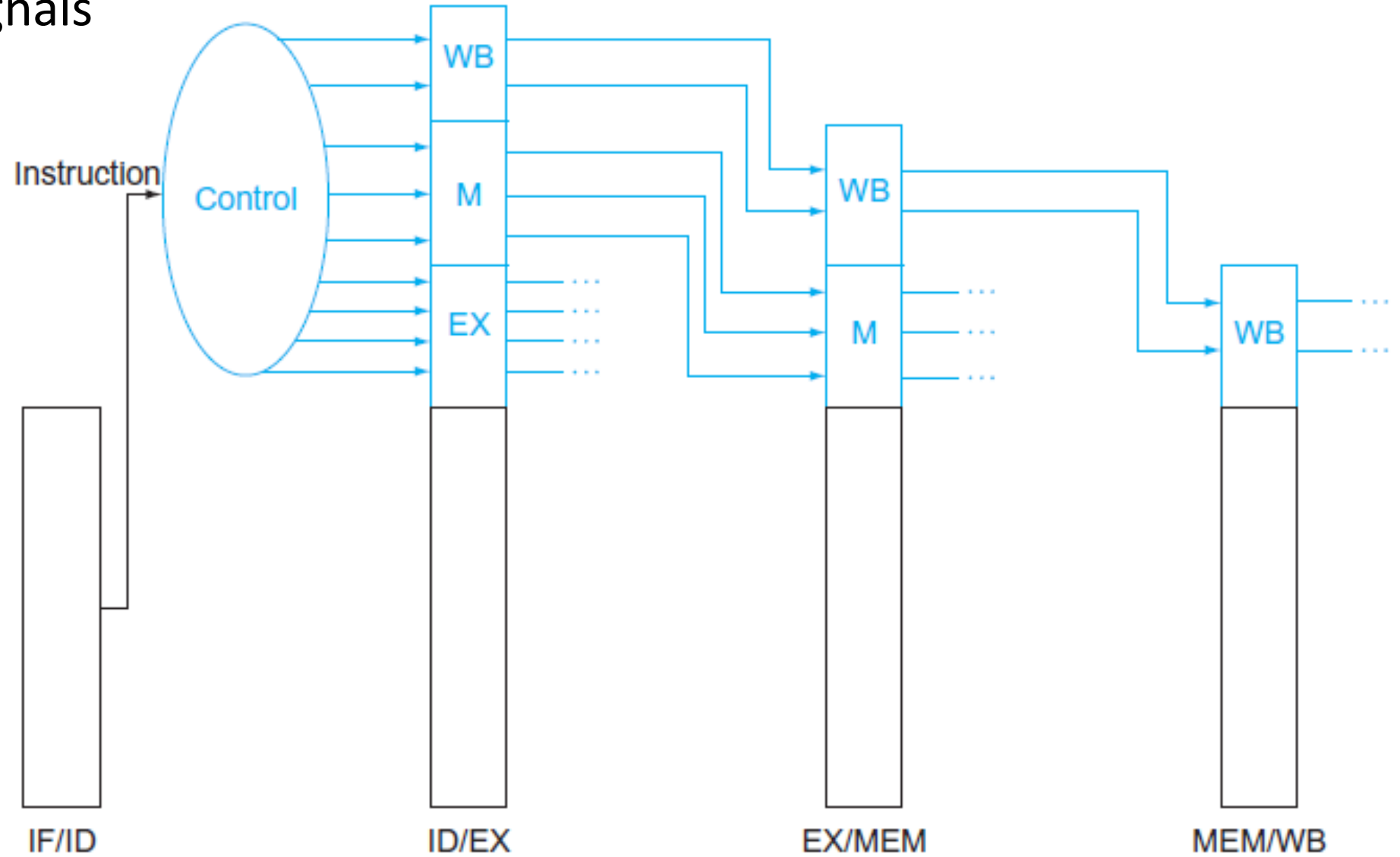


Pipeline – Control Unit

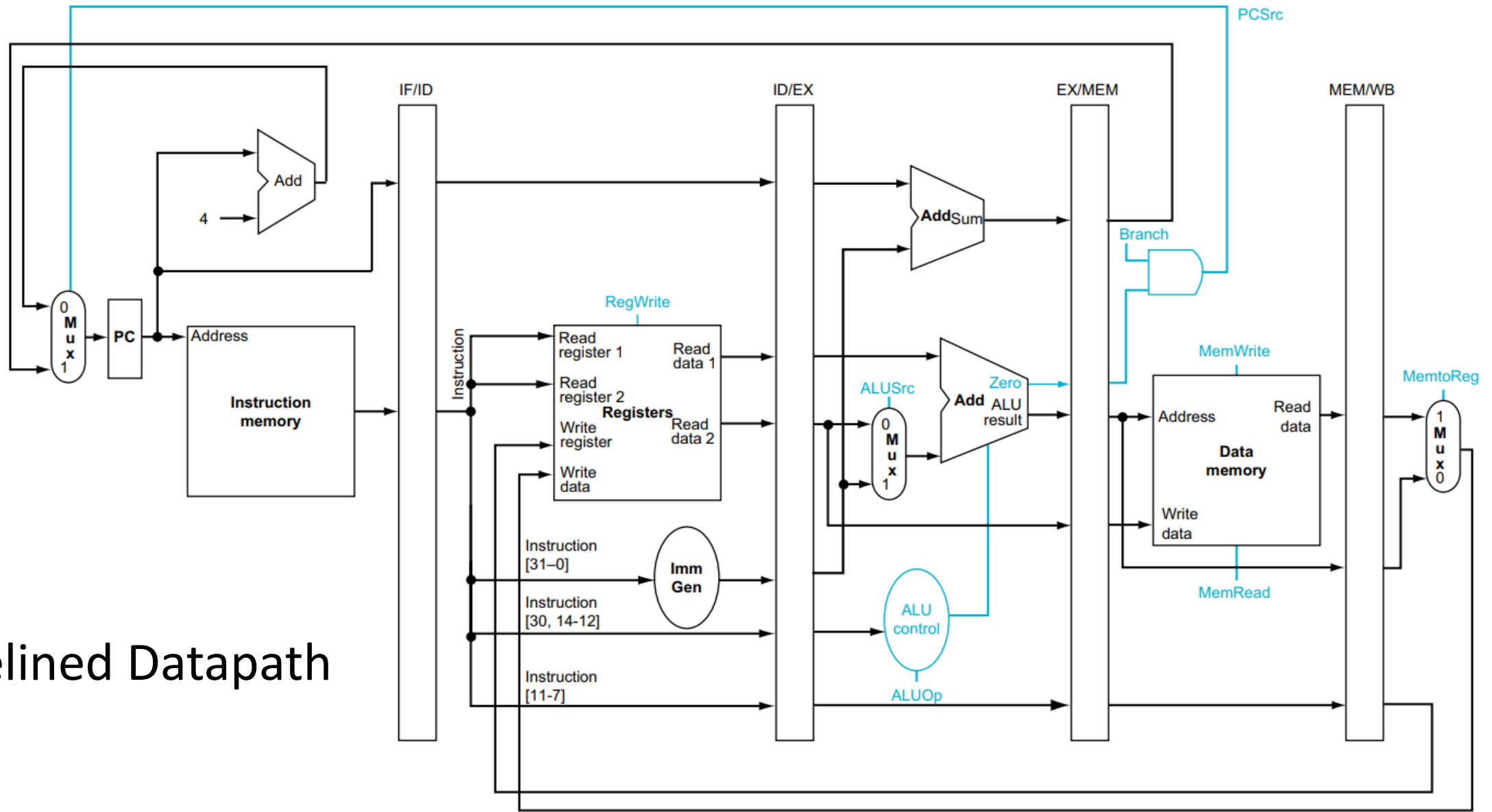


- Same signals as of Single Cycle Processor
- All the control signals will be issued at the same time when the instruction is in the ID stage

Pipeline - Control Signals



Pipelined Datapath

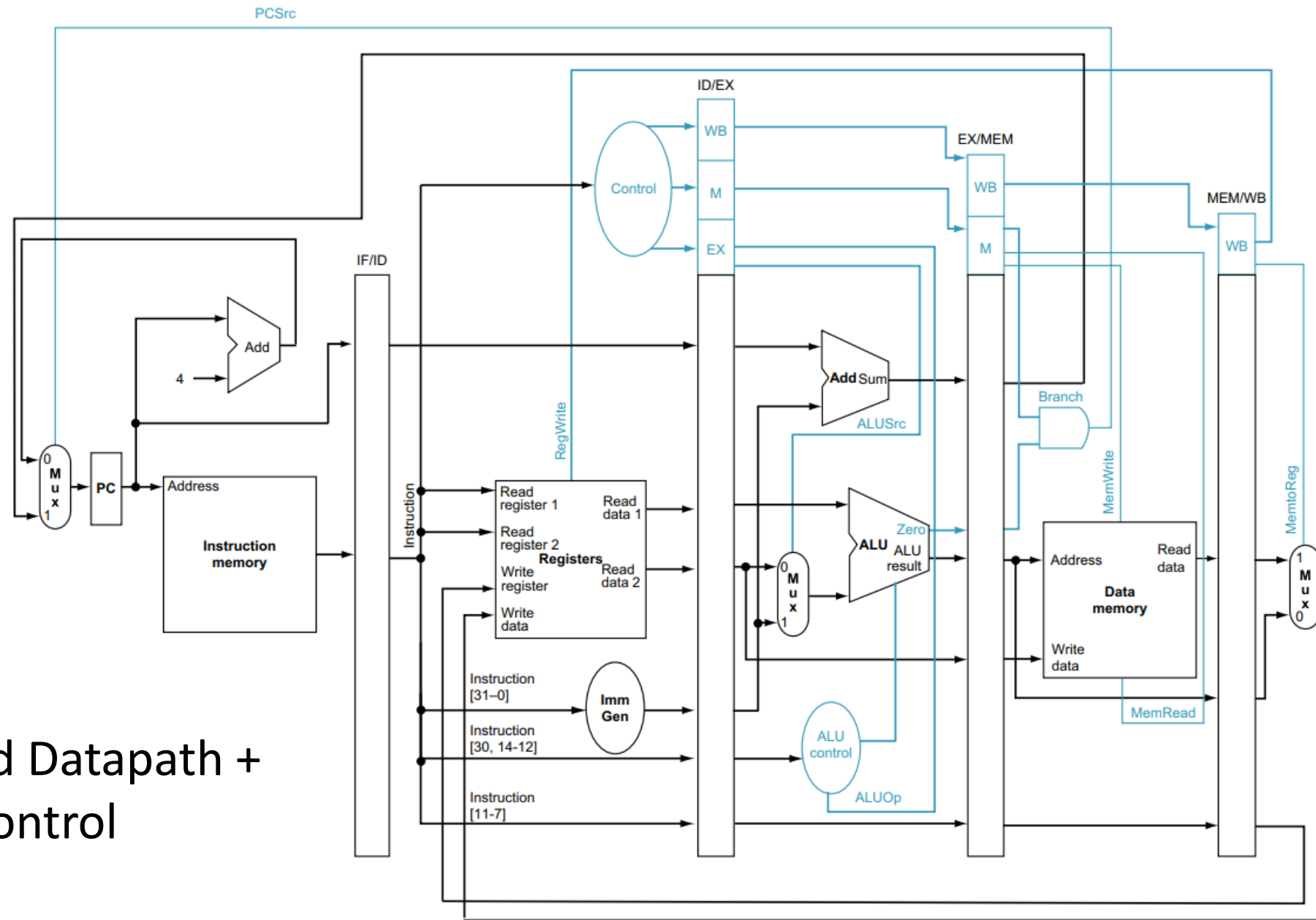


Pipelined Control

Instruction	Execution/address calculation stage control lines		Memory access stage control lines			Write-back stage control lines	
	ALUOp	ALUSrc	Branch	Mem-Read	Mem-Write	Reg-Write	Memto-Reg
R-format	10	0	0	0	0	1	0
lw	00	1	0	1	0	1	1
sw	00	1	0	0	1	0	X
beq	01	0	1	0	0	0	X

Instruction	ALUOp	operation	Funct7 field	Funct3 field	Desired ALU action	ALU control input
lw	00	load word	XXXXXXX	XXX	add	0010
sw	00	store word	XXXXXXX	XXX	add	0010
beq	01	branch if equal	XXXXXXX	XXX	subtract	0110
R-type	10	add	0000000	000	add	0010
R-type	10	sub	0100000	000	subtract	0110
R-type	10	and	0000000	111	AND	0000
R-type	10	or	0000000	110	OR	0001

Signal name	Effect when deasserted	Effect when asserted
RegWrite	None.	The register on the Write register input is written with the value on the Write data input.
ALUSrc	The second ALU operand comes from the second register file output (Read data 2).	The second ALU operand is the sign-extended, 12 bits of the instruction.
PCSrc	The PC is replaced by the output of the adder that computes the value of $PC + 4$.	The PC is replaced by the output of the adder that computes the branch target.
MemRead	None.	Data memory contents designated by the address input are put on the Read data output.
MemWrite	None.	Data memory contents designated by the address input are replaced by the value on the Write data input.
MemtoReg	The value fed to the register Write data input comes from the ALU.	The value fed to the register Write data input comes from the data memory.



Pipelined Datapath + Control

Single Cycle vs Pipeline

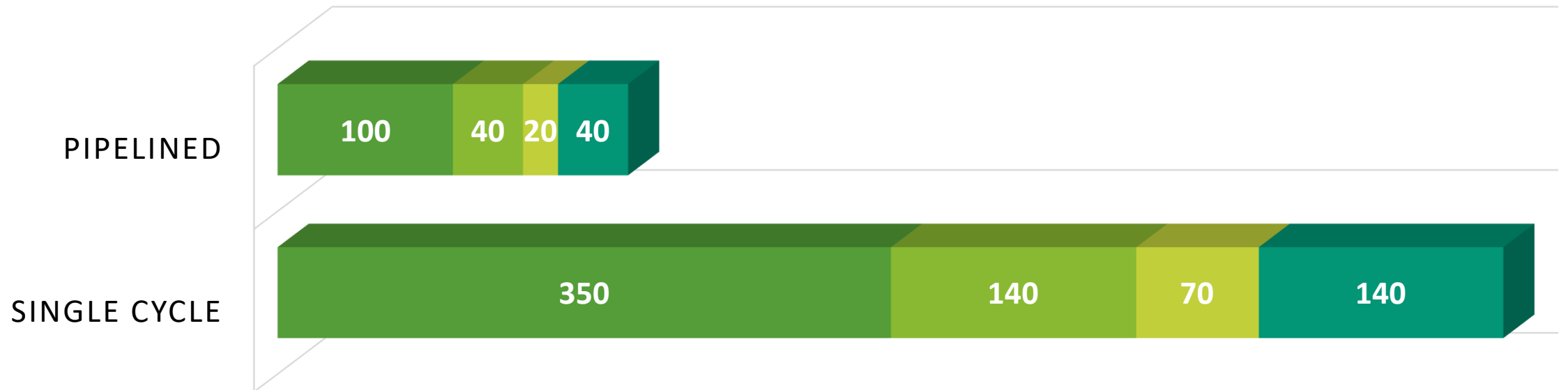
R-type	50%
Load word	20%
Store word	10%
Branch	20%



Instruction Memory	2 units
ALU	1 unit
Register Read/Write	1 unit
Data Memory	2 units

Say we have 100 instructions

■ R-type ■ Load ■ Store ■ Branch



Computer Architecture

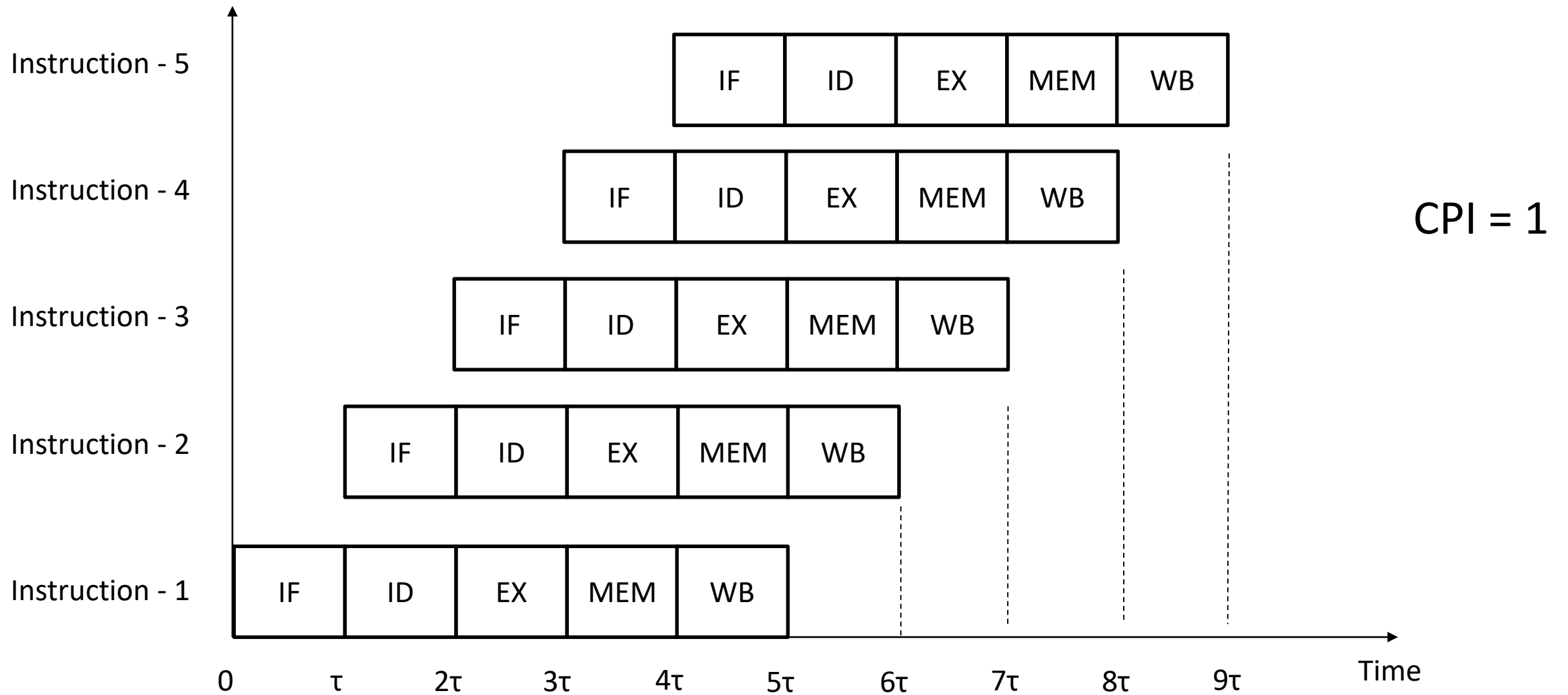
CE/CS 321/330

L2

Course Teacher: Dr.-Ing. Shehzad Hasan

Lecture: 19

Pipeline Hazards



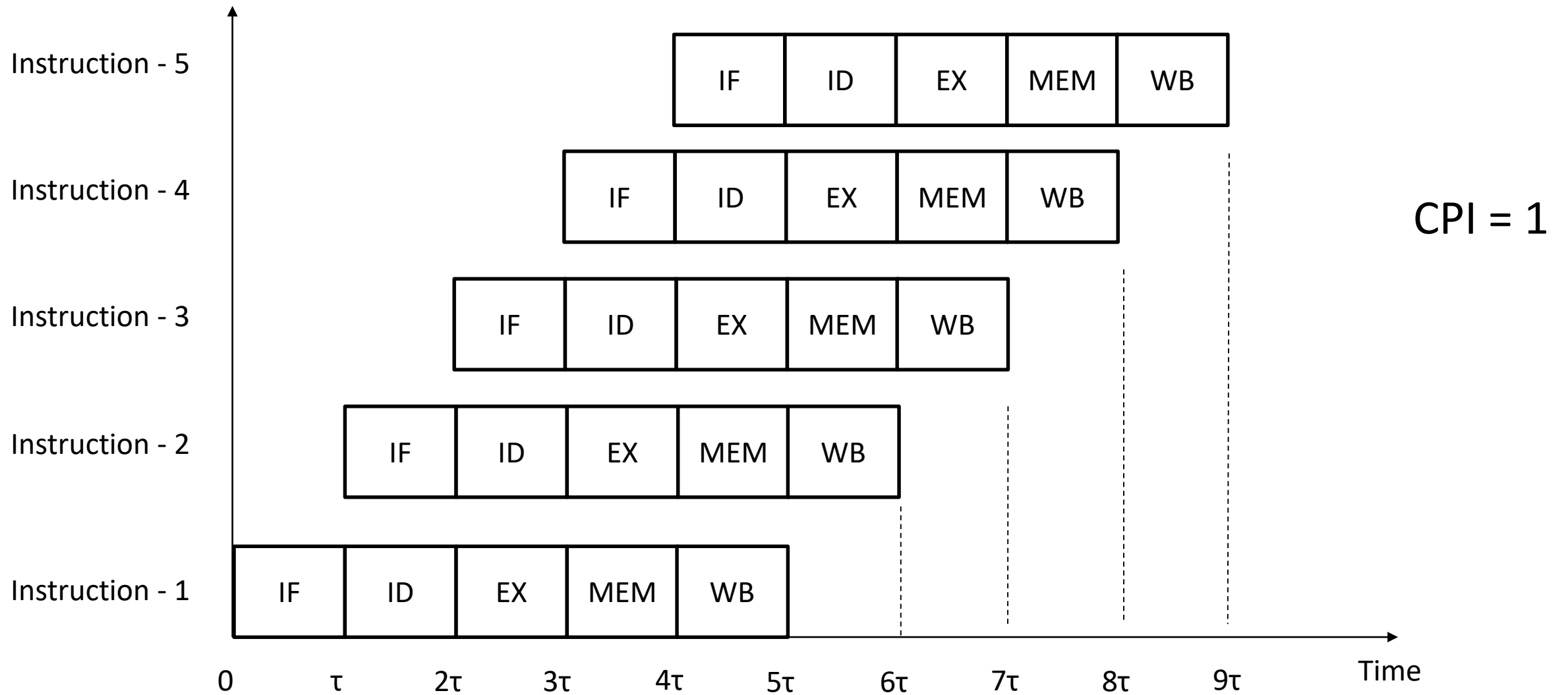
Pipeline Hazards

- There are situations in pipelining when the next planned instruction cannot execute in the following clock cycle. These events are called pipeline hazards.
- The hazards prevent the pipelined processor to achieve the ideal CPI of 1.

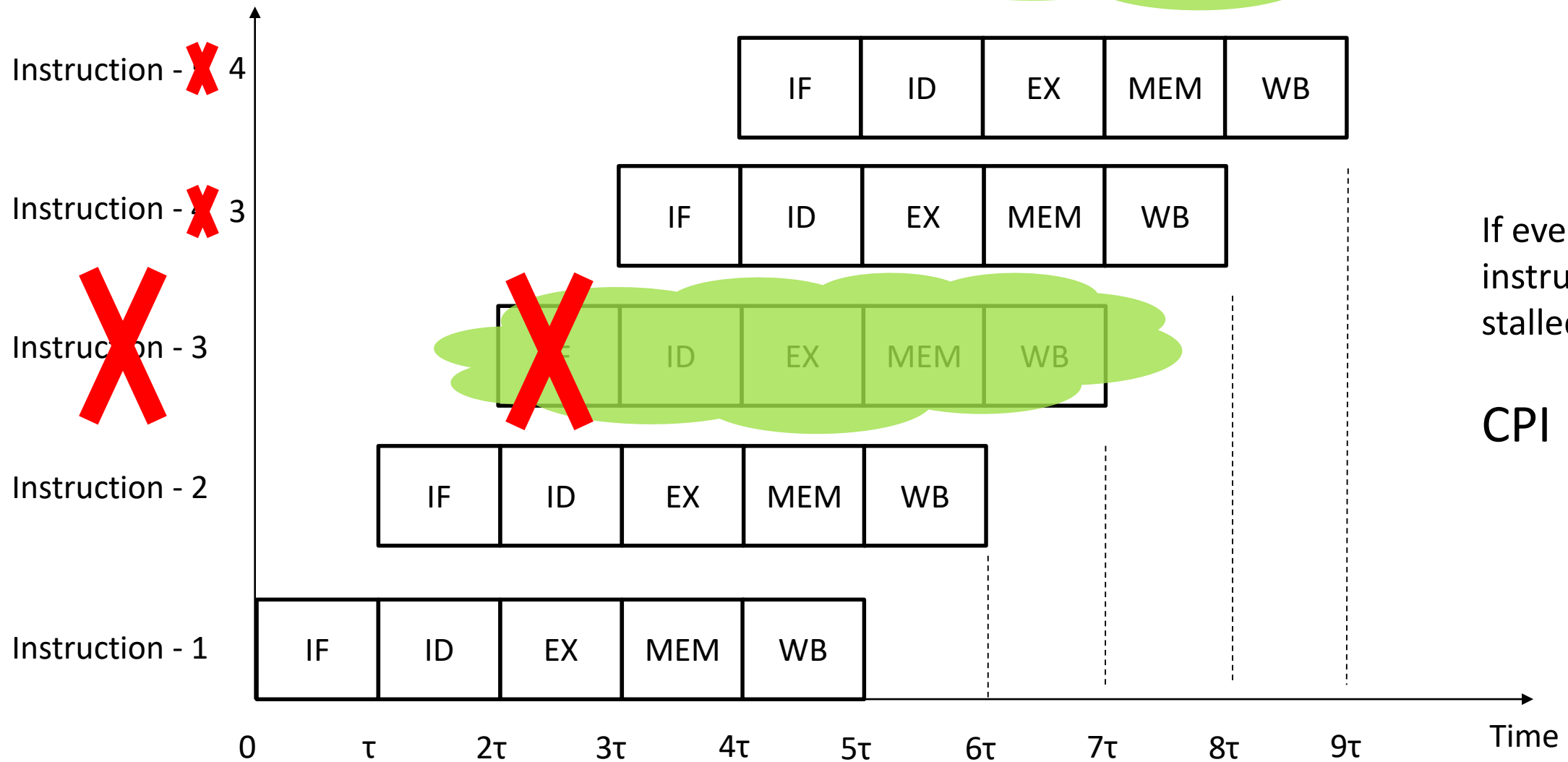

$$\text{CPU time} = \text{IC} \times \text{CPI} \times \text{clock cycle time}$$

- Some hazards can be resolved by changing the pipeline organization otherwise a stall cycle is inserted.

Pipeline Hazards



Pipeline Hazards



If every third instruction is stalled then

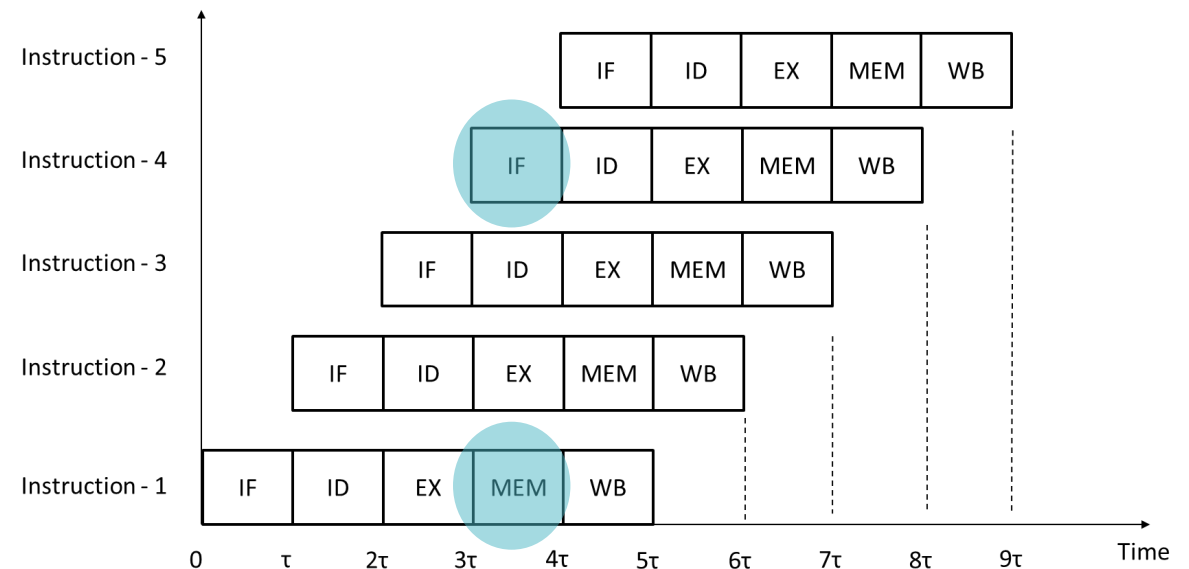
$$\text{CPI} = \frac{3}{2} = 1.5$$

Pipeline Hazards

- There are three types of hazards

1. Structural Hazard

- When a planned instruction cannot execute in the proper clock cycle because the hardware does not support the combination of instructions that are set to execute.
- Resource conflicts, e.g. having a single memory unit and fetching both instruction and data at the same time.

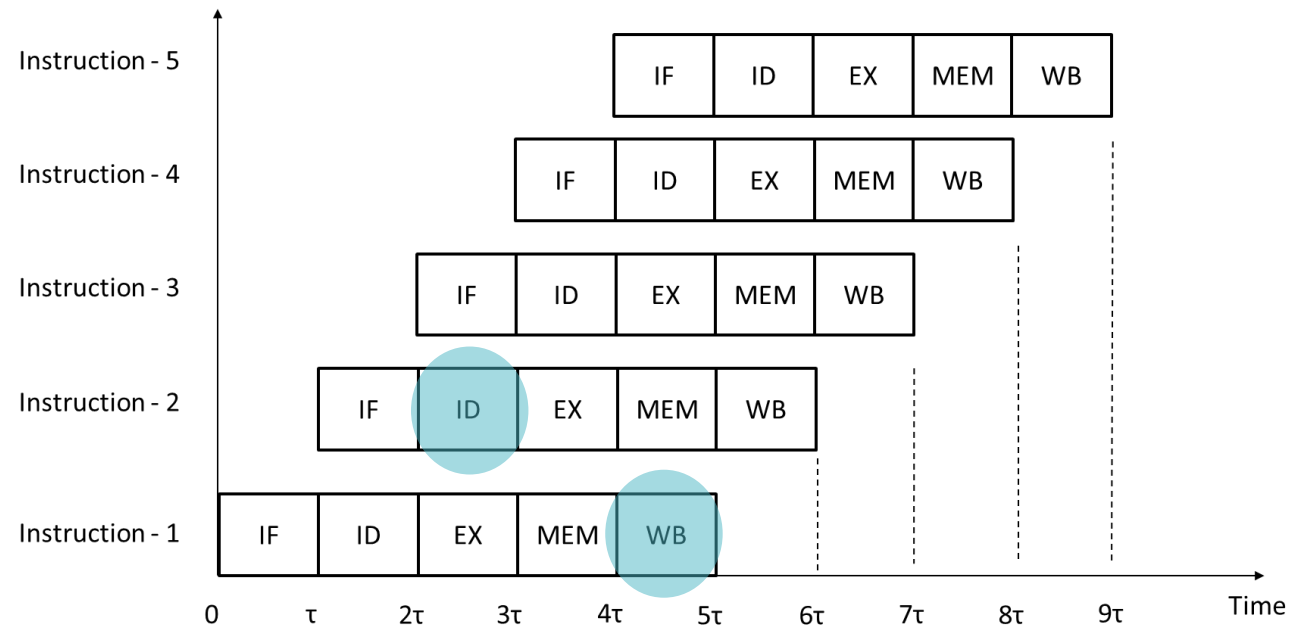


Pipeline Hazards

2. Data Hazards

- When a planned instruction cannot execute in the proper clock cycle because data that is needed to execute the instruction is not yet available.
- Data hazards arise from the dependence of one instruction on an earlier one that is still in the pipeline, e.g.

add x5, x6, x7
sub x8, x9, x5

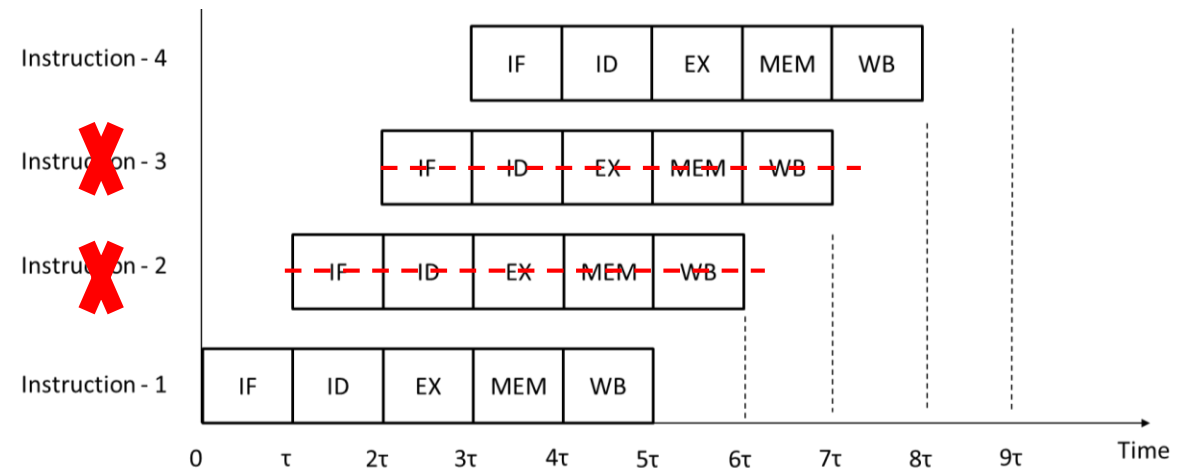


Pipeline Hazards

3. Control Hazards

- When the proper instruction cannot execute in the proper pipeline clock cycle because the instruction that was fetched is not the one that is needed.
 - i.e. The flow of instruction addresses is not what the pipeline expected.
- The cause of control hazards are branch instructions in the program.

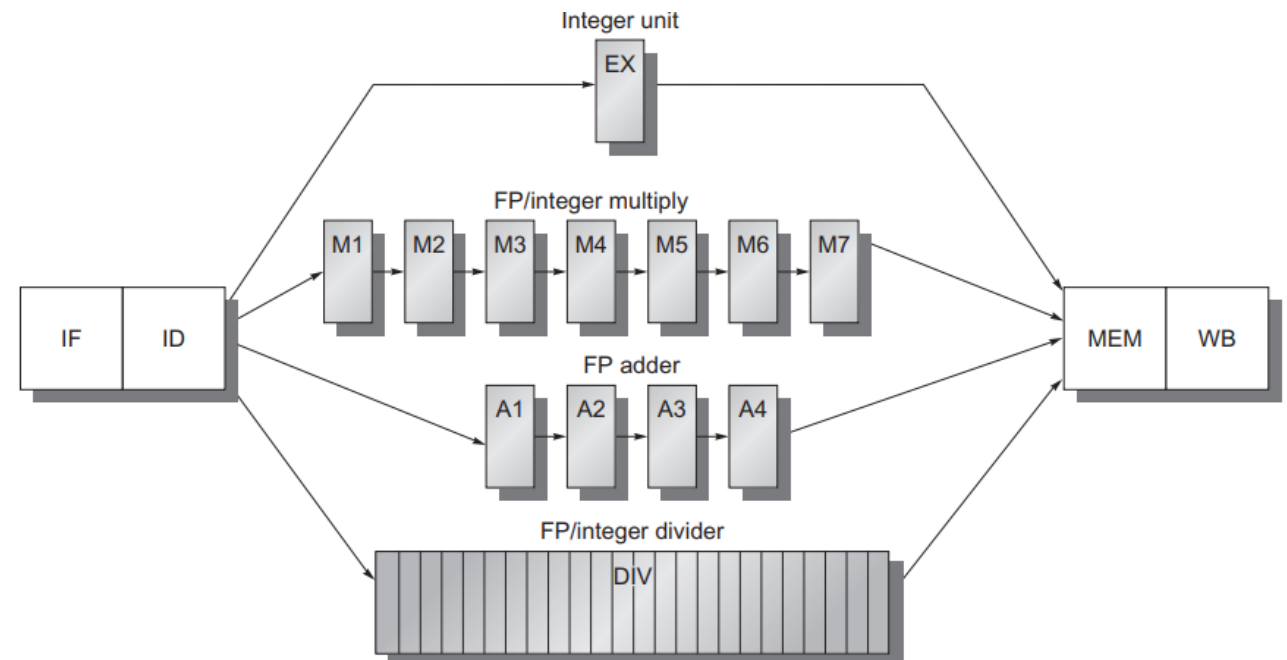
```
top:    lw      x10, 100(x3)
        ⋮
        beq     x5, x5, top
        sub     x8, x9, x4
        add     x7, x9, x4
```



1. Structural Hazards

Reasons

1. When the resource has not been duplicated enough to allow all combinations of instructions in the pipeline to execute.
2. When some functional unit is not fully pipelined.



1. Structural Hazards

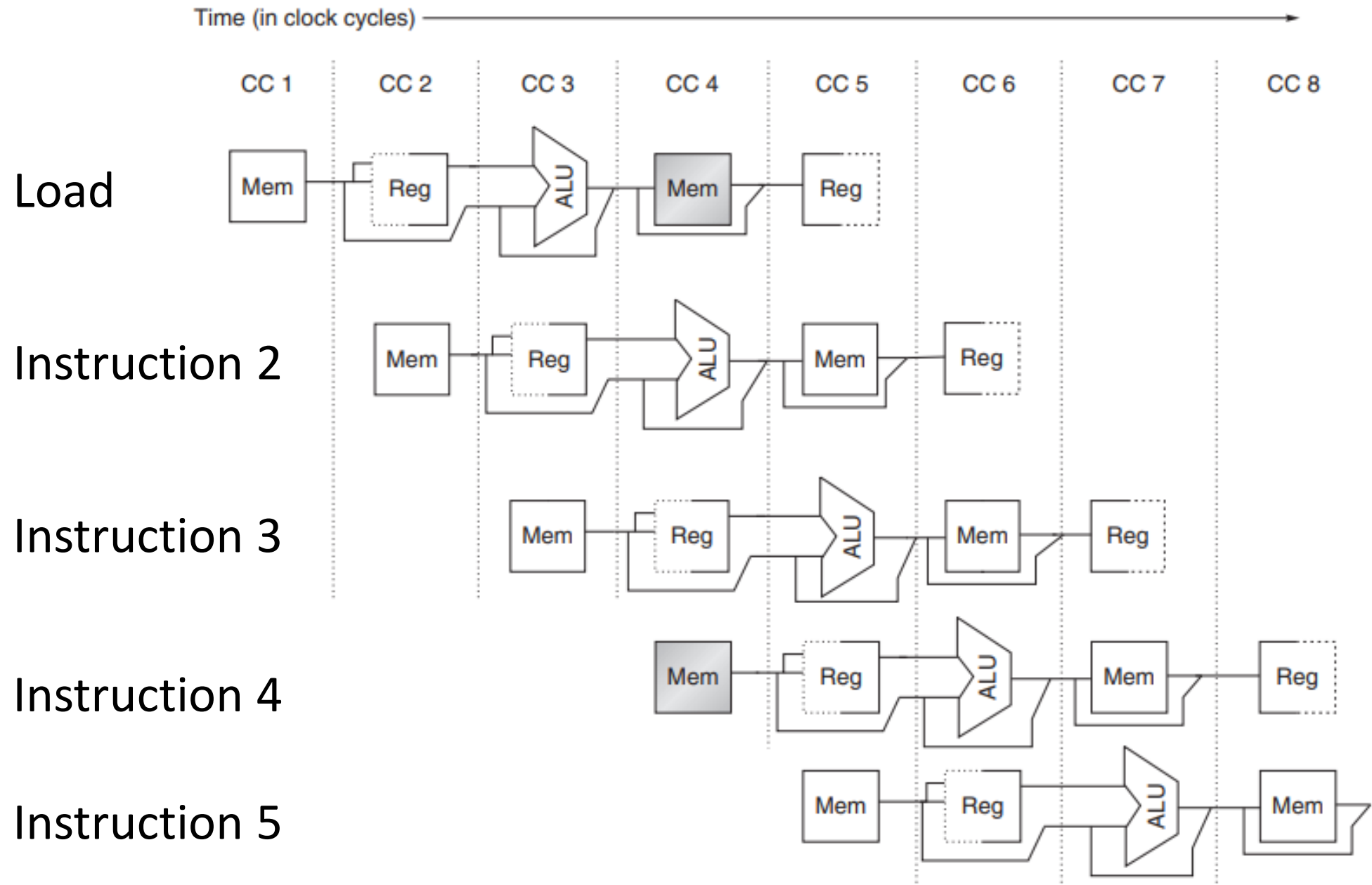
Reasons

1. When the resource has not been duplicated enough to allow all combinations of instructions in the pipeline to execute.
 2. When some functional unit is not fully pipelined.
- When a sequence of instructions encounters this hazard, the pipeline will stall one of the instructions until the required unit is available. Such stalls will increase the CPI from its usual ideal value of 1

Example

- Some pipelined processors have a shared single-memory for data and instructions. As a result, when an instruction contains a data memory reference, it will conflict with the instruction reference for a later instruction.

Structural Hazards



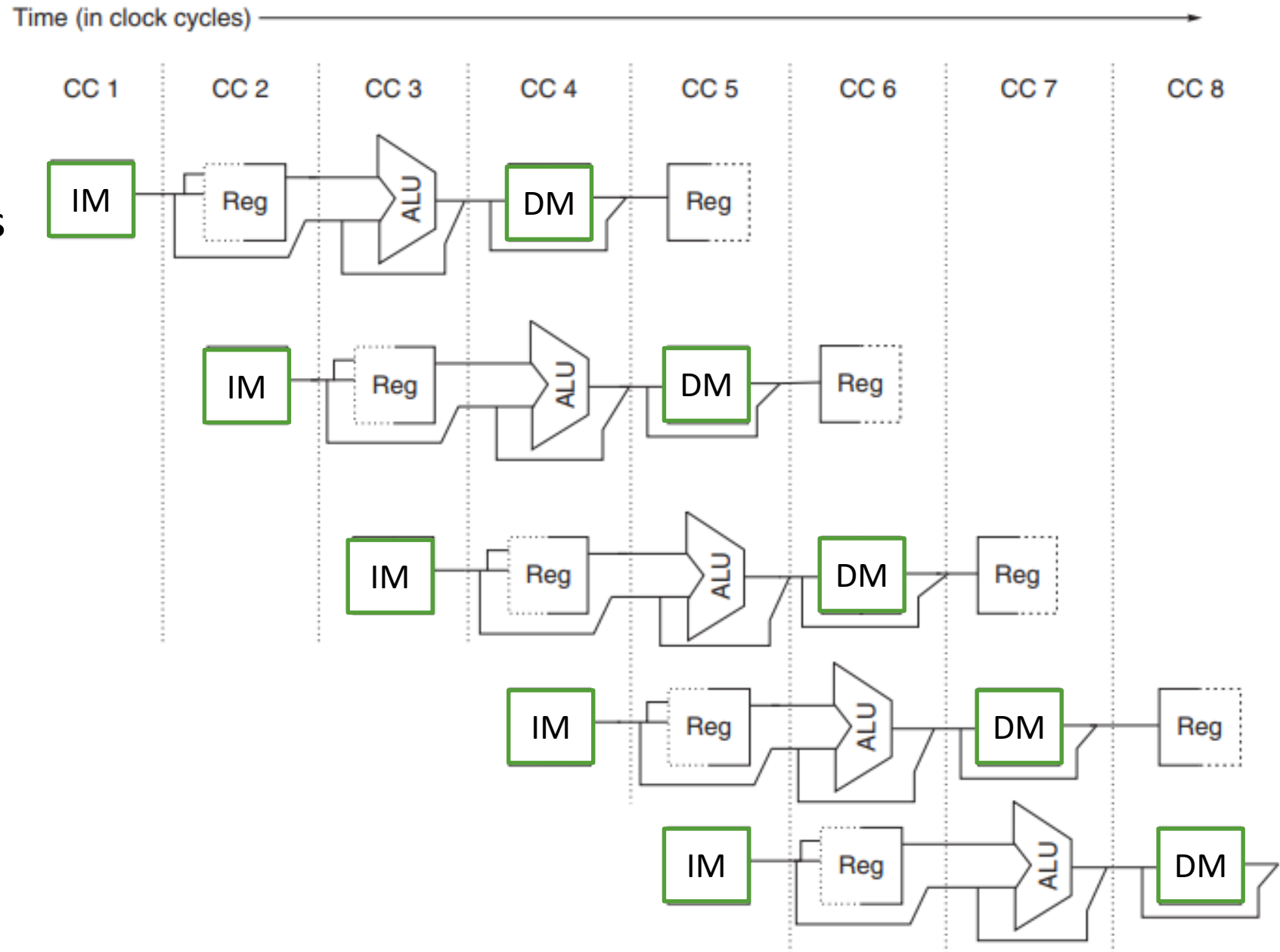
Structural Hazards

- To resolve this hazard, the pipeline is stalled for 1 clock cycle
- A stall is commonly called as a pipeline bubble or just bubble

Instruction	Clock cycle number									
	1	2	3	4	5	6	7	8	9	10
Load instruction	IF	ID	EX	MEM	WB					
Instruction 2		IF	ID	EX	MEM	WB				
Instruction 3			IF	ID	EX	MEM	WB			
Instruction 4				Stall	IF	ID	EX	MEM	WB	
Instruction 5						IF	ID	EX	MEM	WB
Instruction 6							IF	ID	EX	MEM

Structural Hazards

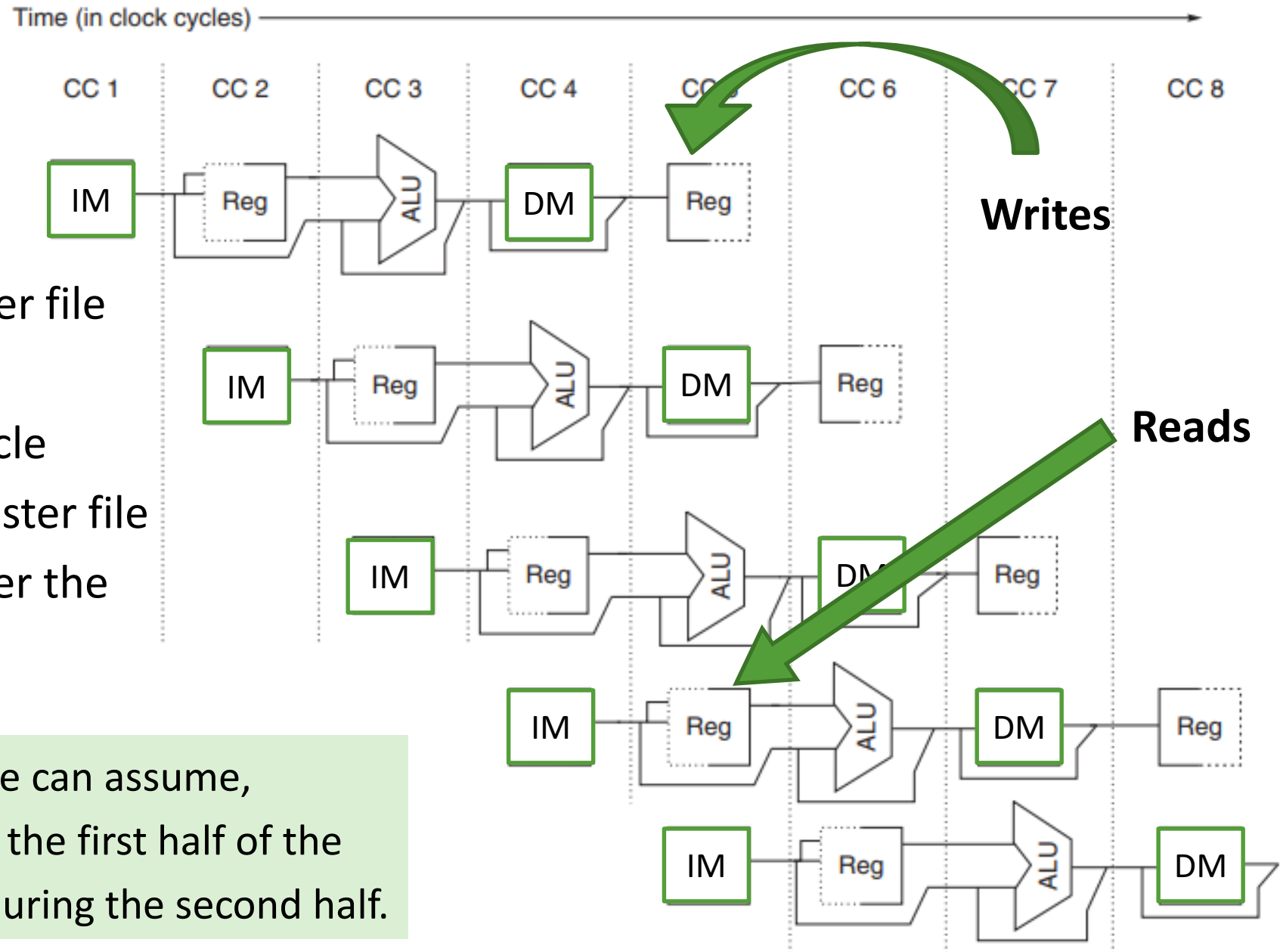
- Stalling the pipeline is not a good solution
- Harvard Architecture, use of different memory units for Instruction and data fetches



Structural Hazards

- Writes to the register file will happen in the beginning of the cycle
- Reads from the register file will be followed after the writes

To keep things simple we can assume, register file is written in the first half of the clock cycle and is read during the second half.

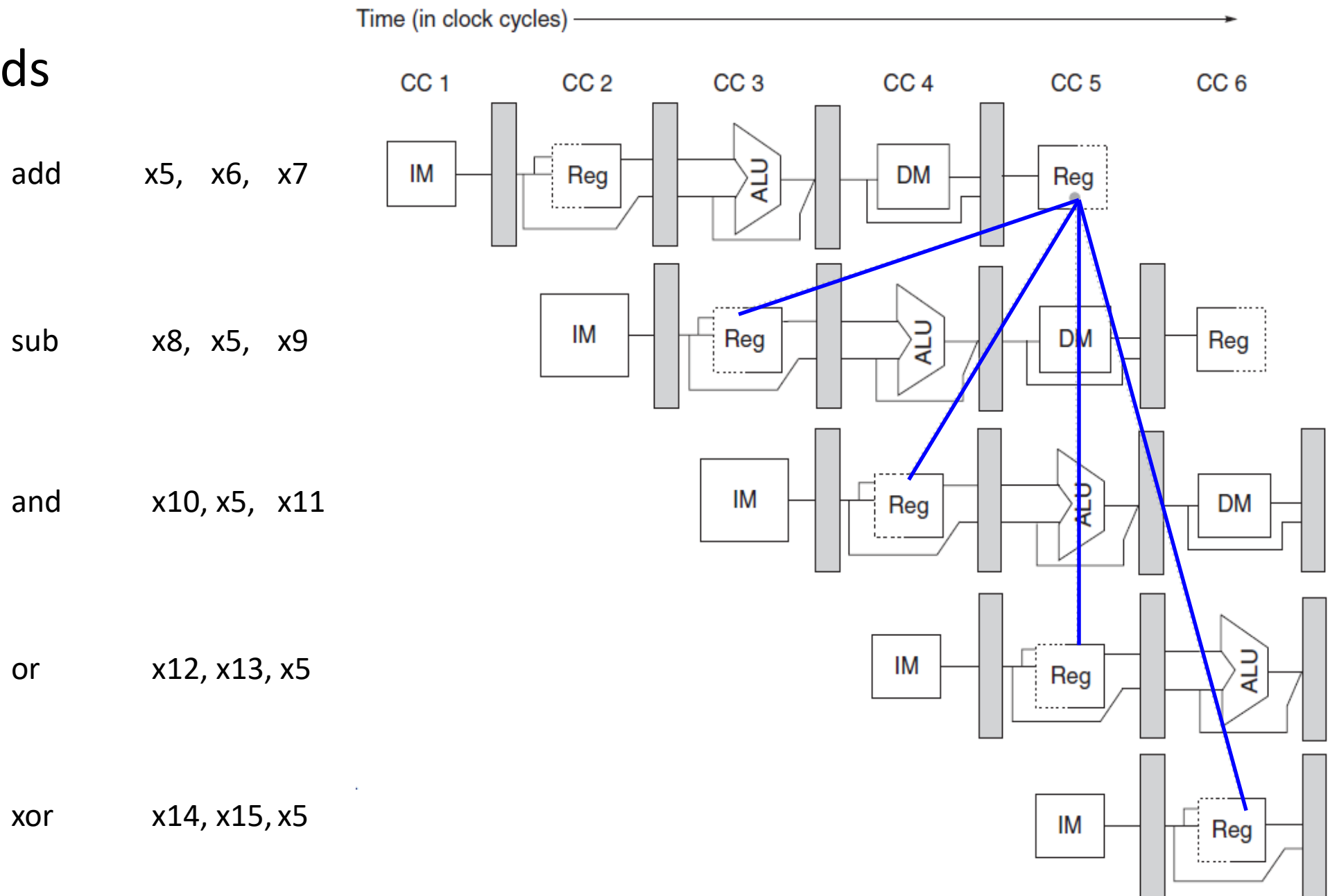


2. Data Hazards

- Data hazards arise from the dependence of one instruction on an earlier one that is still in the pipeline.
- Data hazards occur when the pipeline **changes the order of read/write accesses** to operands so that the order differs from the order seen by sequentially executing instructions on a non-pipelined processor

add	x5,	x6,	x7
sub	x8,	x5,	x9
and	x10,	x5,	x11
or	x12,	x13,	x5
xor	x14,	x15,	x5

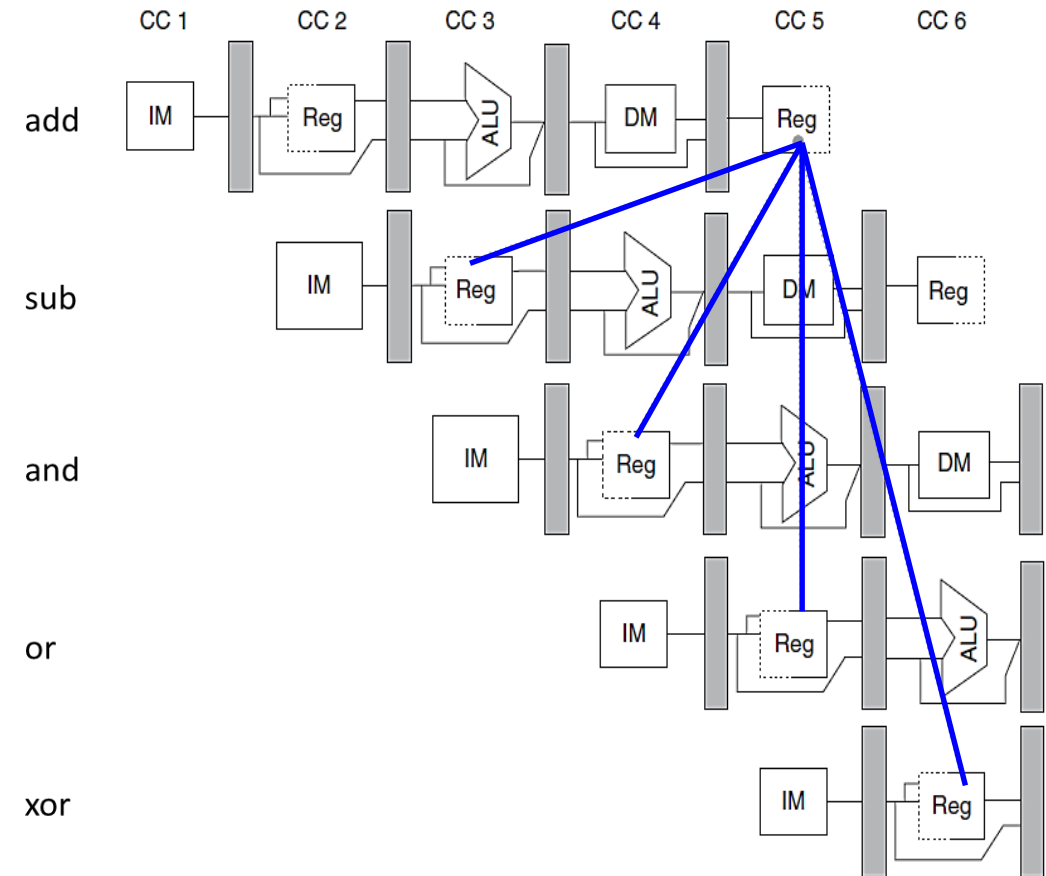
Data Hazards



Data Hazards

There are a number of data dependencies between various pair of instructions as detailed below

- 1) The ADD instruction writes the value of x5 in the WB stage, but the SUB instruction reads the value during its ID stage
- 2) The AND instruction is also affected by this hazard. The write of x5 does not complete until the end of clock cycle 5. Thus, the AND instruction that reads the registers during clock cycle 4 will receive the wrong results.
- 3) The OR instruction operates without incurring a hazard because we perform the register file reads in the second half of the cycle and the writes in the first half.
- 4) The XOR instruction also operates properly because its register read occurs in clock cycle 6, after the register write.



Data Hazards

